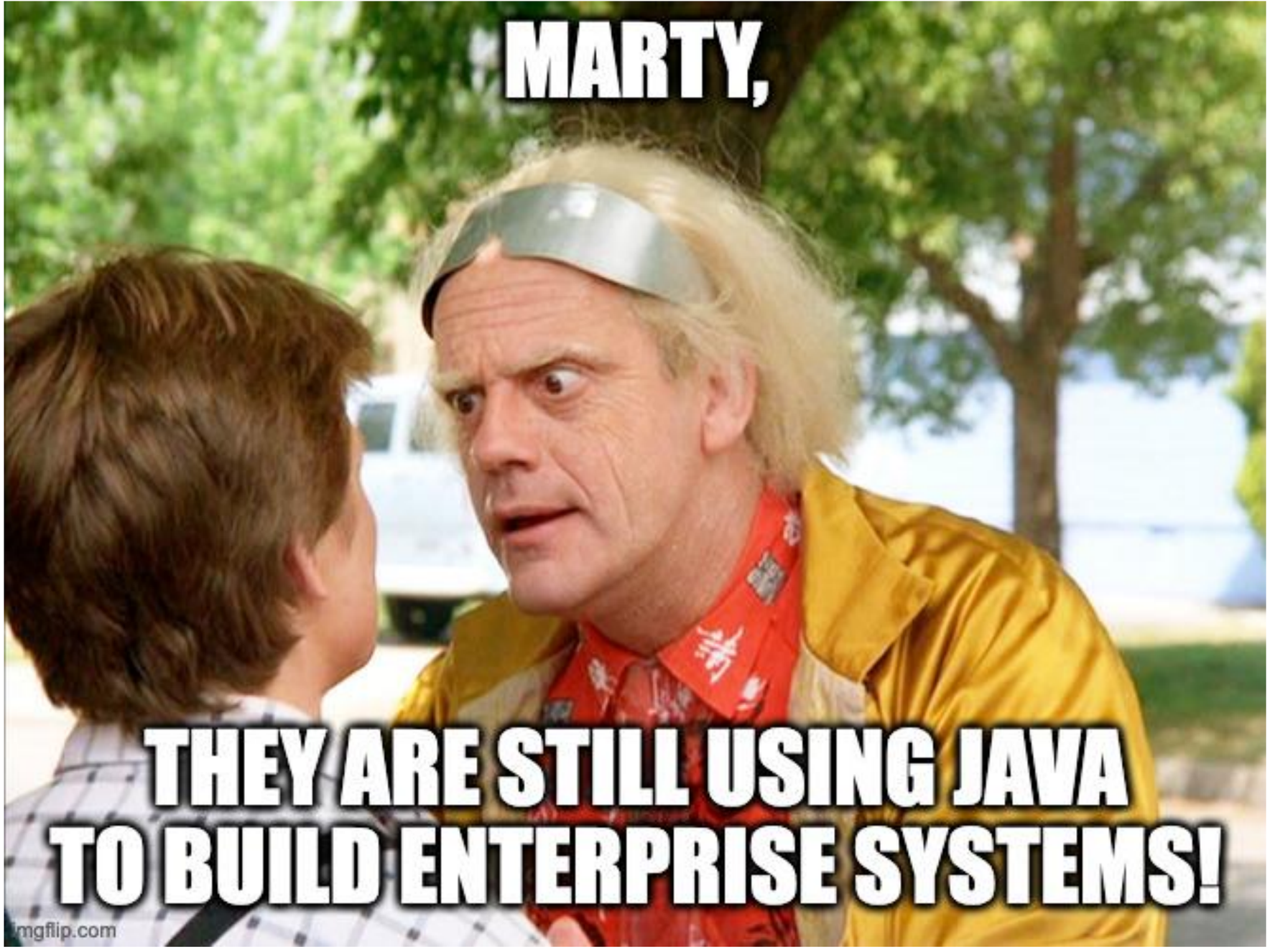
A few things I’d do if I wanted to modernize my Java application in 2026

Áurea Muñoz| IBM
Quarkus Team
Java Champion






MARTY,
THEY ARE STILL USING JAVA
TO BUILD ENTERPRISE SYSTEMS!
imgflip.com

What is modernizing about?

Modernizing isn’t about starting over. It’s about making the next change easier, safer and less expensive.

Things I’d do

1. Ask yourself why, and understand what you have
2. Stop waiting for the perfect moment
3. **Pick a target** that doesn’t force a day-one rewrite
4. Know when to use rules and when to use AI
5. Validate every step

Thing 1

Ask yourself why, and understand what you have

Before you touch any code, ask yourself

1. Why now? What's the trigger?
2. What happens if we do nothing?
3. What was tried before, and why did it fail?
4. Who holds the knowledge?

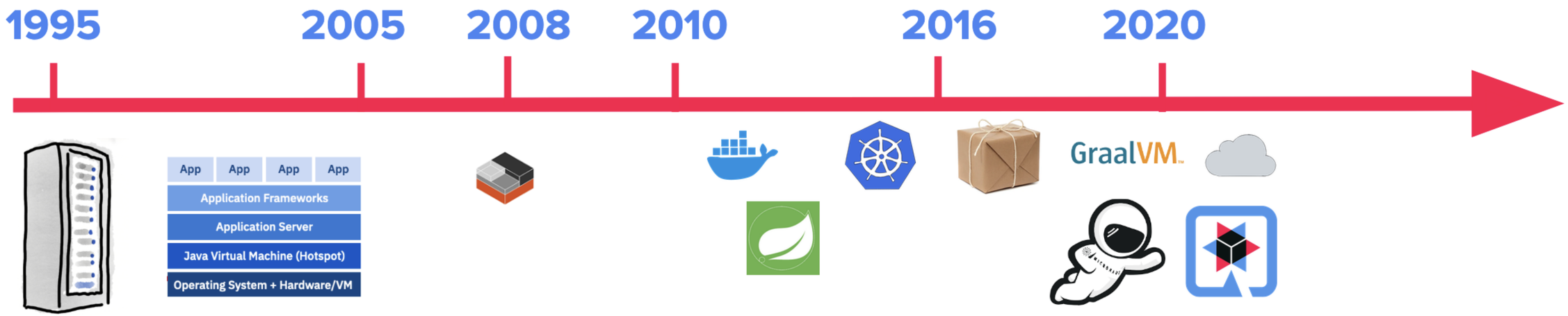
Understand where you are

- Dependencies
- Coupling
- Tests
- Risks

Thing 2

Stop waiting for the perfect moment

From monolithic to containers



Thing 2

The perfect moment doesn’t exist.

Start small. Make progress and validate each step.

Thing 3

Pick a target that doesn’t force a day-one rewrite.

Update your JDK



Incremental over big-bang

Quarkus

Spring compatibility extensions

- quarkus-spring-web, spring-di, spring-data-jpa
- Your Spring code runs as-is
- Migrate to full Quarkus APIs later, or don't

Micronaut

Spring annotation mapping at compile time

- @RestController, @Autowired, Spring Data annotations
- AOT compilation zero runtime overhead
- Build libraries that work on both frameworks

Jakarta EE 11

The ecosystem is converging

- Jakarta Data 1.0: standardized repository pattern
- @Repository, @Find, @Query feels like Spring Data
- Vendor-neutral, compile-time processed

Keep your familiar patterns, change the runtime. The team chooses based on risk and timeline.



QUARKUS

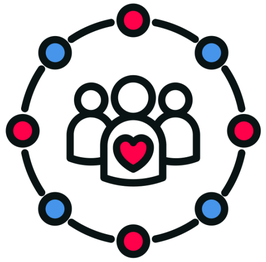
Supersonic Subatomic Java



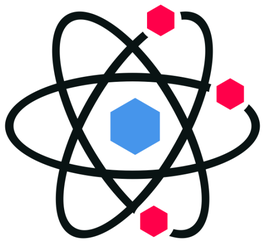
Developer Joy



Kubernetes Native



Best of Breed Libraries



Sequential & Event driven

Thing 4

Know when to use rules and when to use AI.

The boundary

Deterministic rules

- Reproducible
- Cheap (zero tokens)
- Auditable
- Same input = same output

AI

- Flexible
- Expensive (tokens)
- Variable
- Needs context to shine

Quarkus 4

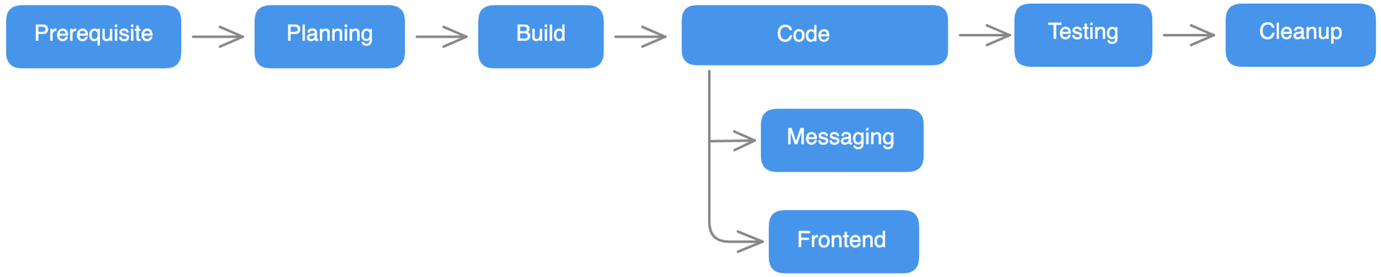
Quarkus 4 GA end of November

Java 21, Hibernate 8, Jackson 3, Jakarta REST 4, CDI 5, Vert.x 5 with HTTP/3

Demo time

```
quarkus update --stream=4.0 --no-rewrite #dry-run
quarkus update --stream=4.0 -y #apply
git diff #review changes
./mvnw install #verify build + tests
```

How we built the Migration skill

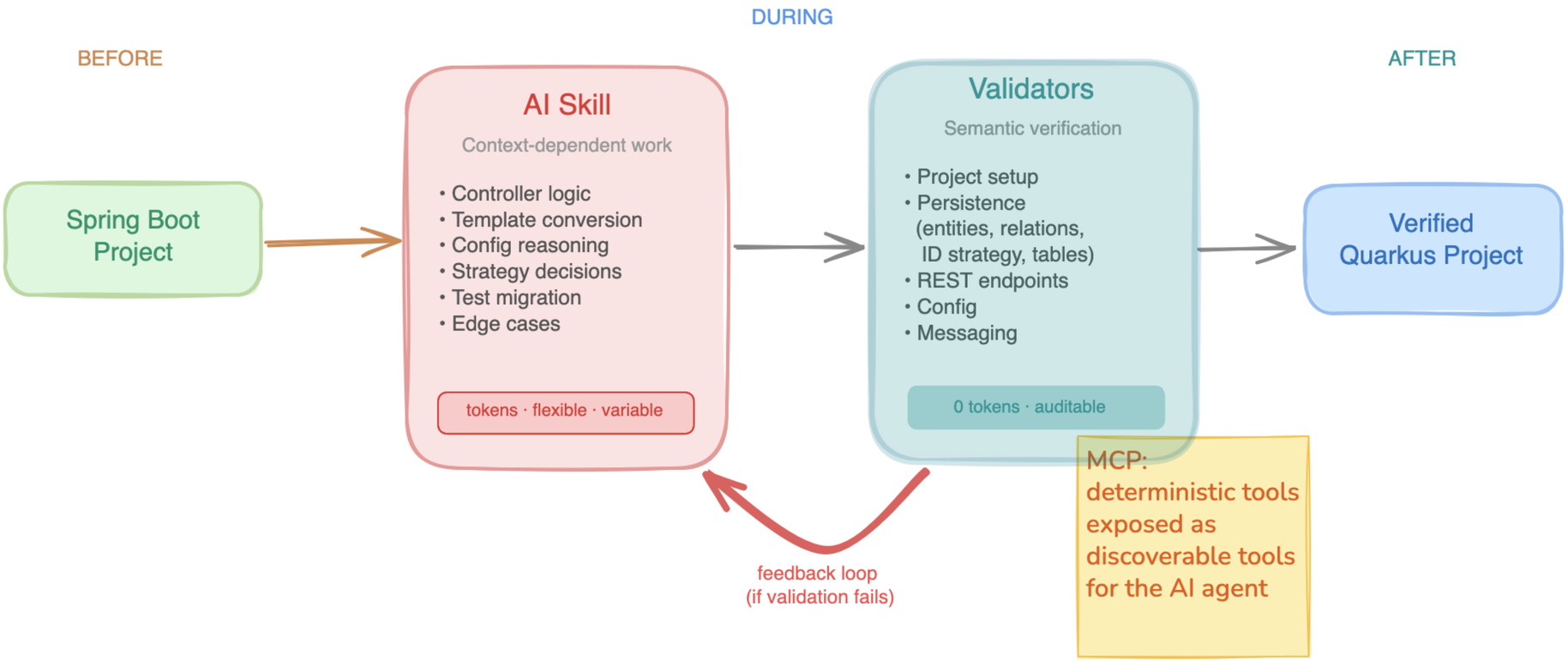


Module	Gate condition	Compile check
Prerequisite	Always — abort if JDK < 17	—
Planning	Always — writes migration-spec.yaml	—
Build	Spring Boot markers in build file	✓
Code	Spring annotations in source	✓
Messaging	@KafkaListener, @RabbitListener, etc.	✓
Frontend	Thymeleaf, JSP, FreeMarker templates	✓
Testing	@SpringBootTest in test sources	✓
Cleanup	Always	✓

Thing 5

Validate every step

Hybrid: deterministic + AI



Demo time

```
claude -p "Read skills/migrate-spring-to-quarkus/SKILL.md \
   and execute it. \
   Source: ~/tmp/gotechworld-demo/spring-petclinic \
   Target: ~/tmp/gotechworld-demo/spring-petclinic-quarkus \
   Strategy: full-quarkus, Java 21, \
   persistence: panache-active-record. \
   Proceed autonomously."
```

Recap

1. Ask yourself why, and understand what you have
2. Stop waiting for the perfect moment
3. Pick a target that doesn’t force a day-one rewrite
4. Know when to use rules and when to use AI
5. Validate every step

Modernizing isn’t about starting over.
It’s about making the next change easier, safer and less expensive.

Mulțumesc

Feedback is welcome

Everything is opensource. Try it, break it, give us feedback.

Slides and demo



Q&A

auri@ibm.com